Room Number _____

Seat Number _____

Student Number

ANONYMOUSLY MARKED

(Please do not write your name on this exam paper)

THE UNIVERSITY OF SYDNEY

CONFIDENTIAL EXAM PAPER

This paper is not to be removed from the exam venue

Computer Science

EXAMINATION

Semester 1 - Final, 2024

COMP2017/COMP9017 Systems Programming

EXAM WRITING TIME: 2 hours

READING TIME: 10 minutes

Questions to complete

COMP2017	COMP9017
Q1-4 (5 marks)	N/A
Q5 (15 marks)	**Q5** (15 marks)
Q6 (50 marks)	**Q6** (50 marks)
Q7 (30 marks)	N/A
N/A	**Q8** (35 marks)
(100 marks)	(100 marks)

EXAM CONDITIONS:
This is a CLOSED book exam - no material permitted

MATERIALS PERMITTED IN THE EXAM VENUE:
(No electronic aids are permitted e.g. laptops, phones)

Calculator – non-programmable

Formula sheet (provided in the exam paper by unit coordinator)

MATERIALS TO BE SUPPLIED TO STUDENTS:
None

INSTRUCTIONS TO STUDENTS:

Do not turn over this page until you are told!

- Please read all the questions first before you begin writing
- Answer all questions in the spaces provided in this question booklet. Do not write your answers in the answer booklets, which are only to be used for drafting and will not be graded.
- Return this question booklet and any additional answer booklets or materials.
- Write your final answers in black or blue ink, not pencil. Take care to write clearly and legibly. Drawing diagrams using pencil is permitted.
- Please write your student number in the space provided at the top of every odd numbered page of this question booklet.
- State clearly on empty pages which question, or part of the question you are completing. There are extra working pages on pages 19-21 of this question booklet.
- Do not remove any tear or remove any pages from the booklet. Staff are not responsible for missing pages in this case. If you find there are missing pages, please alert the examination supervisors.

Please tick the box to confirm that your examination paper is complete. ☐

COMP2017 ONLY. (Multiple choice) (5 marks). 1 mark each unless stated otherwise

---

Question 1 What is the size of a `int *` type, where the size of int is 32 bits and the CPU uses 64 bit addressable memory?

- ☐ 4 bytes
- ☐ 8 bytes
- ☐ 2 bytes
- ☐ 64 bytes
- ☐ 1 byte
- ☐ 32 bytes

Question 2 What is the expected output of the following C code:

```
void foo(int x) {
    if ( x < 5 ) {
        static int y = 5;
        x = y + x;
        printf("%d, ", x);
        y += 1;
    }
}

void main() {
    for (int i = 7; i >= 0; i--)
        foo(i);
}
```

- ☐ `9, 9, 9, 9, 9`
- ☐ `5, 6, 7, 8, 9`
- ☐ Segmentation Fault
- ☐ `9, 8, 7, 6, 5`
- ☐ `7, 6, 5, 4, 3, 2, 1`
- ☐ `5, 6, 7, 8, 9`
- ☐ `0, 1, 2, ... 7`

Question 3 What is a mutex?

- ☐ a synchronisation point where threads block until a certain number of threads have arrived
- ☐ a specific critical section of code that prevents simultaneous access by multiple threads
- ☐ a synchronization object used to allow multiple threads to serialize their access to shared data.
- ☐ a synchronisation point after which threads must continue execution sequentially

Question 4 ♣ - Multiple answer. Worth 2 marks
Which of the following are necessary conditions for a deadlock to occur?

- ☐ did not use a monitor (monitors are more efficient than mutexes and semaphores)
- ☐ hold and wait (threads hold some resources and request other resources).
- ☐ race conditions present (conditions for which code is executed that is contrary to expected behaviour)
- ☐ mutual exclusion (a resource can only be assigned to at most one thread)
- ☐ lack of dynamic memory (no further data can be stored in static/stack memory)
- ☐ circular wait (a cycle exists, each thread waits for a resource that is assigned to another thread)
- ☐ memory leaks present (memory was allocated and not freed correctly address sanitizer)
- ☐ missing compiler flags (almost always necessary for detecting deadlock)
- ☐ no preemption (a resource assigned to a thread can only be released by that thread)
- ☐ has zero edges in a task dependency graph
- ☐ none of the answers are related to deadlock

Question 5 (Memory) (15 marks)

Write the memory contents after execution of all statements up to line 10. Write the names of each variable, its address and any data associated with them in the appropriate box.

```
1  static char val = -47;
2  int main(int argc, char **argv) {
3      char *str = "XYZA";
4      char tr = str[2];
5      char *sp = &val;
6      char x = *sp + 2;
7      char **holder = malloc(sizeof(char*));
8      *holder = &val;
9      tr = tr + **holder + 2;
10         ; // show snapshot of memory at this statement
```

The starting address for each segment of memory has been provided for you. For the purpose of this exam, you should assume that memory is allocated positively from 0x0000 to 0xFFFF.

`sizeof(char *)` is 4 bytes. Assume no word alignment for memory.
ASCII value `'A'` is 65. Assume memory does not align to nearest word size.

Consider these two lines of code, the memory contents may look like ⟶

```
char a = 5;
char *b = &a;
```

Sample memory		
0x7000	a	5
0x7000+1	b	0x7000

Program code
0x2000

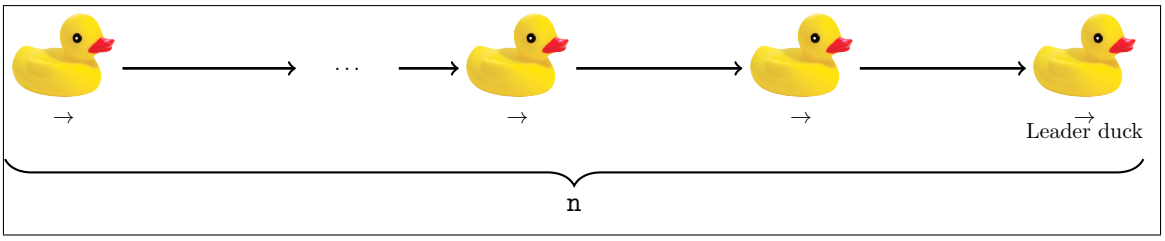
Static/Global
0x3000

Heap
0x4000

Stack
0x7000

Question 6 (Programming) (50 marks)

You are to complete part of a simulation for `n` ducks in flight. The ducks fly in a single line formation in 1D space. Each has an x coordinate and velocity and a reference to the duck in front.



All ducks at the **beginning** of the simulation (above). All ducks are facing the same direction with velocity `1.0f`. Ducks positions are represented as float, and positions are not equally spaced apart. The velocity value is both speed and direction (positive or negative).

The following are definitions used in the simulation.

```
typedef int (*quack_f)(int);

struct duck {
    float position[1]; // x
    float velocity[1]; // x in m/s
    struct duck *duck_in_front; // the duck in front
    quack_f quack;
};

extern quack_f happy_quack, sad_quack, angry_quack, neutral_quack;

extern int same_direction(struct duck *, struct duck *);
extern void update_position(struct duck **, int, float, int *);
```

In the simulation, events happen first, then the duck positions are updated, similar to the code below. Your task is to write the functions `same_direction()` and `update_position()`.

```
BEGIN
    initialise_sim(...)
    DO
        event_duck_thinking( ... )
        event_world( ... )

        update_position( ... ) // YOUR TASK

        render( ... )
        sim_control( ... )

        t += timestep
    LOOP UNTIL end
END
```

Part 1: Write the code for the `same_direction` function - 5 marks

```
/* returns >0 if both ducks move in the same direction
 * returns 0 if either ducks velocity is zero
 * returns <0 otherwise
 * if d1 or d2 are NULL, the behaviour is undefined
 */
int same_direction(struct duck *d1, struct duck *d2)
```

Parts 2 - 5 are implemented within the function. Each part is implemented in order. You are to write the entire code for function:

```
void update_position(struct duck **ducks, int n, int max, float timestep, int *died)
```

The arguments are:

- `ducks`, an array of `struct duck` pointers with size `max`, and `n` active ducks. Assume non-`NULL` elements are allocated on heap. $0 <= n <= max$ greater than zero.
- `timestep` the amount of time passed in the current step of the simulation. Assume always larger than `FLOAT_EPS`.
- `died` a pointer to one integer to indicate how many ducks have died. Assume non-`NULL`.

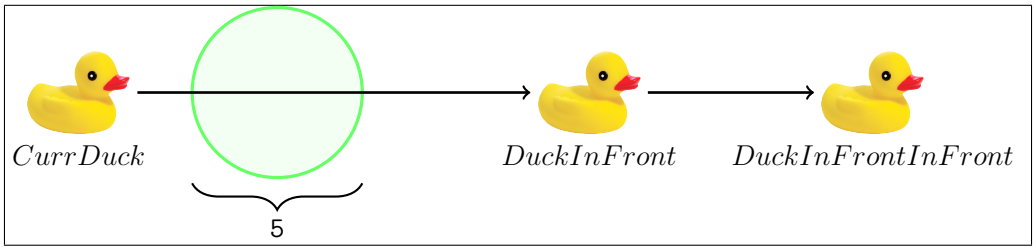
Part requirements: In each part (2 - 5), write code as part of `update_position()`. Each part operates on all ducks using the order presented in the array. The examples denote the current duck as `CurrDuck`. `CurrDuck`'s non `NULL` `duck_in_front` reference is labelled `DuckInFront`. There is only one leader of the flock in a given direction, positive or negative. The `ducks` array can have an element whose value is `NULL`, this element should be skipped. You cannot duplicate the O(n) memory of the ducks to solve these problems.

Part 2: Update duck position - 13 marks

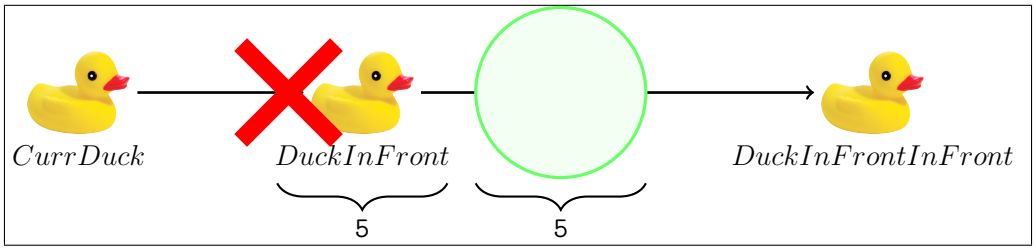
Refer to *Part requirements*. Update each duck struct's position $\mathbf{p}$ according to the velocity $\mathbf{v}$ and the timestep Δt.

$$\mathbf{p}' = \mathbf{p} + \mathbf{v}\Delta t$$

The duck cannot occupy the area of another duck. If the duck's new position $\mathbf{p}'$ is within radius **2.5**, then the duck will iteratively attempt to move to position $\mathbf{p}' + \mathbf{vk}$ until there is no other duck within the radius. Here, $\mathbf{k}$ is an integer greater than 0.



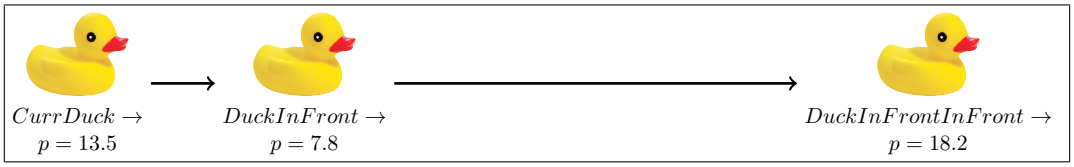
Part 2: Position update. CurrDuck can move to green circle located at $\mathbf{p}'$ without issue.



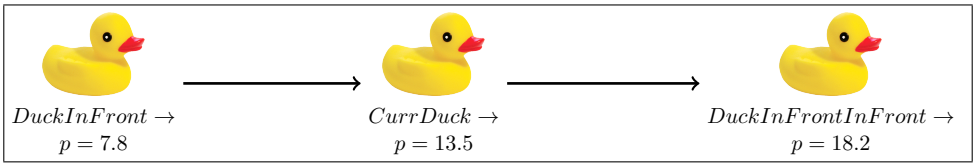
Part 2: Position update with clash. CurrDuck cannot occupy the next position located at $\mathbf{p}'$. They can occupy $\mathbf{p}' + \mathbf{v} \times 1$, where no other duck is within radius 2.5.

Part 3: Overtaking duck - 13 marks

Refer to *Part requirements*. Overtaking occurs when `CurrDuck` and `DuckInFront` have the same direction (based on `same_direction()`) and `CurrDuck`'s new position after Part 2 is further than `DuckInFront`'s position *in that direction*. In this case, the relationship of duck in front is modified. `DuckInFront`'s `duck_in_front` becomes `CurrDuck`. `CurrDuck`'s `duck_in_front` becomes `DuckInFrontInFront`. After an overtake has occurred, the `quack` function for `CurrDuck` is set to `angry_quack`, and for `DuckInFront` is set to `sad_quack`. The rule is repeated until there are no more ducks overtaking. Note: overtaking can require revisiting *any* previously processed duck in this part for re-evaluation. State any assumptions you have made about revisiting.



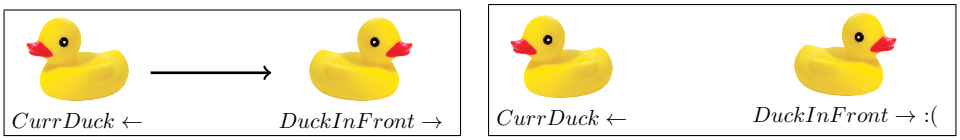
Part 3: Three ducks. *CurrDuck* must overtake due to position and direction requirements.



Part 3: After overtaking

Part 4: Flying separate ways - 6 marks

Refer to *Part requirements*. Modify each duck based on the rule. If `CurrDuck` and `DuckInFront` do not have the same direction (based on `same_direction()`), `CurrDuck` should have it's `duck_in_front` reference changed to `NULL`. `DuckInFront` will then have it's quack function as `sad_quack`.



Part 4: Two ducks flying in a *different* direction (left). Rule applied (right).

Part 5: Dead duck - 13 marks

Refer to *Part requirements*. A duck becomes a dead duck whenever the absolute value of it's velocity is less than 0.005. The memory associated with the dead duck is deallocated and the array element of `ducks` is set to `NULL`.

If any duck has it's `quack` function as `sad_quack` *and* it's `duck_in_front` reference is a dead duck, it will die also. This rule is to be repeated until all ducks who refer to dead ducks and have a `sad_quack` are all dead. Ducks still alive with references to dead ducks should change their `duck_in_front` to `NULL`.

If the number of dead ducks is equal to zero, all ducks have their `quack` function as `happy_quack`, otherwise it is set to `neutral_quack`. `died` should be zero initially and incremented for each dead duck found. Note: dead ducks can require revisiting *any* previously processed duck in this part for re-evaluation.

Notes for all parts

You do not need to consider handling these cases:

- two ducks can occupy the same position within the given radius
- there is no need to maintain the order of the array
- the function has to have each part performed in order. i.e. Part 4 must happen before Part 5.

In your answer, clearly label your code with Part 2, Part 3, etc.

You can implement any helper functions as needed. Useful function:
`double fabs(double x); - absolute value of floating-point number .`

Begin your function here

```
void update_position(struct duck **ducks, int n, int max, float timestep, int *died)
```

Q6 Continue your answer (please clearly label part 2, 3, 4, 5)

Q6 Continue your answer (please clearly label part 2, 3, 4, 5)

Q6 Continue your answer (please clearly label part 2, 3, 4, 5)

Q6 Continue your answer (please clearly label part 2, 3, 4, 5)

Q6 Extra page for this question (please clearly label part 2, 3, 4, 5)

COMP2017 ONLY. Question 7 (Processes) (30 marks)

A program will create n processes $P_0, P_1, P_2, ..., P_i, ..., P_{n-1}$. P_0 is the first process created by execution of the program. P_i is the parent process of P_{i+1}. The hierarchy of process construction cannot be changed. n is known at the beginning of the program.

Describe a method of interprocess communication where one message is transferred from any one process to a specific process using anonymous pipes. For example, if there are 8 processes, process P_6 should be able to send a message to P_3. You are free to construct and arrange the pipes between processes. Only anonymous pipes are allowed; you cannot use shared memory, named pipes, or other file descriptors (sockets, files).
In your answer you must:

i 8 marks - Explain how all the pipes are setup for all processes that are constructed. Write the function `void create_children(...)` and describe which process(es) would call this function.

ii 10 marks - Explain how the messages are encoded and decoded by sending, receiving and even processed by intermediate processes.

Write the function `void send_message( int process, char *msg, int msg_len )` which will initiate sending a message from one process to a specific `process`.

Write the function `void receive_message( char *buffer, int *nread, int buffer_max)` that will be called by a processes when data is available to read on their pipe(s).

iii 4 marks - Write the function `void free_resources(...)` that will be called by a process after receiving a `SIGTERM` signal.

iv 4 marks - Describe how your method prevents cycles in message sending. Using the above example, the originating process P_6 should not receive the message it sends.

v 4 marks - Describe how the message sending operation converges. Using the above example, the destination process P_3 should receive the message exactly once. Other processes involved should only process the message once.

The messages you send must include 50 bytes for the message (`char msg[50];`). For example: `"Reload Data"`

You have the freedom to describe any data that is used in the pipe to communicate to all other processes and extend send/receive message functions to include additional parameters.

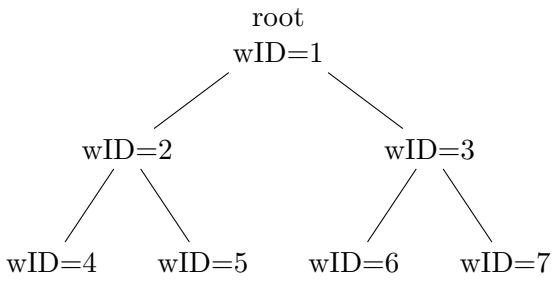
Functions may be written in pseudocode or C code (most accurate), with descriptions in plain English. If your answer is not clear, marks can be deducted. Including the header files are not required.

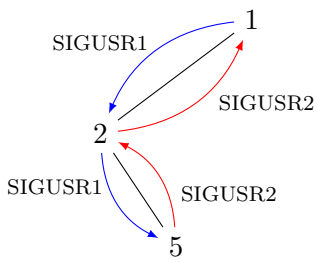
COMP9017 ONLY. Question 8 (Processes) (35 marks)

This program consists of a hierarchical tree of concurrent processes that communicate using only signals. Each child process is a leaf or non-leaf in the tree.

Part 1 - 15 marks

Write a function `int initialise(int n)` to create processes as a complete binary tree of n levels. The root process is the root of the binary tree. Two children of a process are left and right. The number of levels is specified by the integer n. A parent of two children must maintain references of the two children Process ID (PID). On success of initialising the process tree, 0 is returned. You should aim to minimise the resources required for each process. The traversal and labelling of processes worker ID (wID) is shown in the figure:





Left: The hierarchy assumes that a parent creates two children whose numbering is $2 \times parent's\ wID$ and $(2 \times parent's\ wID) + 1$. This 3 level tree has 7 processes. Leaf processes are wID 4,5,6,7. Non-leaf processes are wID 1,2,3. Right (part 2): To communicate with leaf node 5, signals must be propagated from the root node and returned.

Part 2 - 20 marks

Describe an encoding and protocol for *reliably* communicating to leaf processes in the tree using signals from the root. There is no direct signalling between root and leaf. The root will call a function `void issue_command(enum command cmd, uint8_t leaf_wID)`. The communication should only:

- send SIGUSR1 to children (down the tree),
- send SIGUSR2 to parents (up the tree), and
- Each signal sent after a specially encoded 32 bit `int`, designed by you, has been set. This is shared memory with `mmap()` among all processes. `static uint32_t *shared;`.

Commands may include a parameter in addition to the `wID`:

- `RESUME rval` - implementation defined. rval is 4 bits.
- `CHANGE cval` - implementation defined. cval is 8 bits.
- `STOP` - implementation defined.

Assumptions:

- `enum command { RESUME, CHANGE, STOP};`
- valid n values $3 \leq n \leq 7$. Thus an upper limit of the processes in the tree is 127 ($2^n - 1$).
- `static uint32_t *shared;` has already been setup with `mmap()` and is usable directly by each process.
- signals cannot be queued, out of order, or be missed. Be sure to describe these error cases and how you can, or cannot, achieve reliability.

Functions may be written in pseudocode or C code (most accurate), with descriptions in plain English. If your answer is not clear, marks can be deducted. Including the header files are not required.

COMP2017 answer Q7. COMP9017 answer Q8

COMP2017 answer Q7. COMP9017 answer Q8. Additional space

COMP2017 answer Q7. COMP9017 answer Q8. Additional space

End of the Exam

End of Exam extra space

End of Exam extra space

End of Exam extra space

Common types: `int`, `float`, `short`, `char`

Common escape characters: `\n` `\t` `\'` `\"` `\\` `\0`

Array notation: `[ ] *` **Address operator:** `&`

Keywords

`switch` – code jump for equality to constant expressions

`do while` – Loop at least once

`static` – (outside function) does not export symbol outside C module

`static` – (inside function) allocates persistent memory of a variable

`void` – non scalar type

`void *` – pointer to non scalar type

Control Flow Syntax:

- if (...) ... [else if (...) ...] [else ...]
- while (...) ...
- do ... while (...);
- for (... ; ... ; ...) ...
- switch (...) { case x: ...[break;] ...default: ...}

Arithmetic Operators: `+ - * /`

Logical Operators: `! && || == != >= <= < >`

Bitwise Operators: `& | ^ << >>`

Assignment Operators: `= += *= -= /= ?:`

Useful functions:

`void *malloc(size_t size);` - allocates `size` bytes and returns a pointer to the allocated memory.

`void free(void *ptr);` - frees the memory space pointed to by `ptr`

`void *calloc(size_t nmemb, size_t size);` - allocates memory for an array of `nmemb` elements of `size` bytes each and returns a pointer to the allocated memory. The memory is set to zero.

`void *realloc(void *ptr, size_t size);` - changes the size of the memory block pointed to by `ptr` to `size` bytes. The contents will be unchanged in the range from the start of the region up to the minimum of the old and new sizes. If the new size is larger than the old size, the added memory will not be initialized. If `ptr` is `NULL`, then the call is equivalent to `malloc(size)`, for all values of size; if size is equal to zero, and ptr is not `NULL`, then the call is equivalent to free(ptr). Unless `ptr` is `NULL`, it must have been returned by an earlier call to `malloc()`, `calloc()`, or `realloc()`. If the area pointed to was moved, a `free(ptr)` is done.

`void *memcpy(void *dest, const void *src, size_t n);` - copies `n` bytes from memory area `src` to memory area `dest`. The memory areas must not overlap.

`void *memset(void *s, int c, size_t n);` - fills the first `n` bytes of the memory area pointed to by `s` with the constant byte `c`. Returns a pointer to the memory area `s`.

`FILE *fopen(const char *pathname, const char *mode)` – mode `r` Read, `w` Write, `a` Append

`int fclose(FILE *stream)` – Closes the file

`char *fgets(char *s, int size, FILE *stream)` – Reads all input up until a new line or size-1 chars

`int fflush(FILE *stream)` – Writes data stored in the buffer to the file immediately

`int strncmp(const char *s1, const char *s2, size_t n);` – return 0 if `s1` is equal to `s2`, < 0 if `s1` smaller, or > 0 if `s2` smaller. Compares at most `n` characters.

`char *strncpy(char *dest, const char *src, size_t n);` – copies a string from `src` to `dst` for at most `n` characters.

`size_t strlen(const char *s);` – Calculate length of C string, excluding terminating `NULL` byte

`int sscanf(const char *str, const char *format, ...);` – scan input string `str` according to string `format`. Conversions of format string are stored in the location pointed to by the pointer arguments that follow `format`.

C format string

`%c` – character

`%d` – (optionally) signed integer

`%s` – C string

`%256s` – truncation of C string

`%f` – single precision float

`%.3f` – single precision float to 3 decimal places

`void qsort(void *base, size_t nmemb, size_t size, int (*compar)(const void *, const void *));` - sorts an array with `nmemb` elements of size `size`. The base argument points to the start of the array. The contents of the array are sorted in ascending order according to a comparison function pointed to by `compar`, which is called with two arguments that point to the objects being compared. The comparison function must return an integer less than, equal to, or greater than zero if the first argument is considered to be respectively less than, equal to, or greater than the second.

`pid_t fork(void);` - creates a new process by duplicating the calling process. On success, the PID of the child process is returned in the parent.

`int execv(const char *pathname, char *const argv[]);` - replaces the current process image with a new process image. The `const char *arg` and subsequent ellipses can be thought of as `arg0, arg1, ..., argn`. Together they describe a list of one or more pointers to `NULL`-terminated strings that represent the argument list available to the executed program.

`int pipe(int fildes[2]);` - creates a pipe (an object that allows unidirectional data flow) and allocates a pair of file descriptors. The first descriptor connects to the read end of the pipe; the second connects to the write end. `ssize_t read(int fd, void *buf, size_t count);` - attempts to read up to `count` bytes from file descriptor `fd` into the buffer starting at `buf`.

`ssize_t write(int fd, const void *buf, size_t count);` - writes up to `count` bytes from the buffer pointed to by `buf` to the file referred to by the file descriptor `fd`.

`int select(int nfds, fd_set *readfds, fd_set *writefds, fd_set *exceptfds, struct timeval *timeout);` – monitor multiple file descriptors, waiting until one or more of the file descriptors become "ready" for some class of I/O operation (e.g., input possible). A file descriptor is considered ready if it

is possible to perform a corresponding I/O operation.

`void FD_CLR(int fd, fd_set *set);` - clear single descriptor descriptor from set

`int FD_ISSET(int fd, fd_set *set);` - check single descriptor active from set

`void FD_SET(int fd, fd_set *set);` - set single descriptor active from set

`void FD_ZERO(fd_set *set);` - zero entire descriptor set `<sys/wait.h>`

`pid_t wait(int *wstatus);` - wait for state changes in a child of the calling process, and obtain information about the child whose state has changed. `pid_t waitpid(pid_t pid, int *wstatus, int options);` - suspends execution of the calling thread until a child specified by pid argument has changed state. `typedef void (*sighandler_t)(int);`

`sighandler_t signal(int signum, sighandler_t handler);` - sets the disposition of the signal signum to handler, which is either `SIG_IGN`, `SIG_DFL`, or the address of a programmer-defined function (a "signal handler"). Returns the previous value of the signal handler, or `SIG_ERR` on error. In the event of an error, `errno` is set to indicate the cause.

`int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);` - change the action taken by a process on receipt of a specific signal. The sigaction structure is defined as:

```
struct sigaction {
      void        (*sa_handler)(int);
      void        (*sa_sigaction)(int, siginfo_t *, void *);
      sigset_t    sa_mask;
      int         sa_flags;
      void        (*sa_restorer)(void);
};
```

`int kill(pid_t pid, int sig);` - send any signal to any process group or process. On success (at least one signal was sent), zero is returned. On error, -1 is returned, and `errno` is set appropriately.

`int sem_init(sem_t *sem, int pshared, unsigned int value);` - initializes the unnamed semaphore at the address pointed to by sem. The value argument specifies the initial value for the semaphore. Returns 0 on success; on error, -1 is returned, and `errno` is set to indicate the error.

`int sem_destroy(sem_t *sem);` - destroys the unnamed semaphore at the address pointed to by sem. Returns 0 on success; on error, -1 is returned, and `errno` is set to indicate the error.

`int sem_wait(sem_t *sem);` - decrements (locks) the semaphore pointed to by sem. If the semaphore's value is greater than zero, then the decrement proceeds, and the function returns, immediately. If the semaphore currently has the value zero, then the call blocks until either it becomes possible to perform the decrement. Returns 0 on success, otherwise -1 on error and semaphore is unchanged. `int sem_trywait(sem_t *sem);` - is the same as `sem_wait()`, except that if the decrement cannot be immediately performed, then call returns an error (`errno` set to `EAGAIN`) instead of blocking.

`int sem_post(sem_t *sem);` - increments (unlocks) the semaphore pointed to by sem. Returns 0 on success; on error, the value of the semaphore is left unchanged, -1 is returned, and `errno` is set.

`int sem_getvalue(sem_t *sem, int *sval);` - places the current value of the semaphore pointed to sem into the integer pointed to by sval.